

Bayes' Rule and Bayesian Inference

Table of Contents

1. [Introduction](#)
 2. [Bayes' Rule Formula](#)
 3. [Terminology: Prior, Likelihood, Posterior](#)
 4. [Why Bayes' Rule Matters](#)
 5. [Working with Bayes' Rule](#)
 6. [Sequential Bayesian Updates](#)
 7. [Practical Examples](#)
 8. [Common Pitfalls](#)
-

Introduction

Bayes' Rule is one of the most important formulas in probability and statistics. It allows us to update our beliefs based on new evidence.

Real-world example: Finding the lost submarine

- **Before search:** We have some idea where it might be (prior)
 - **After search:** We get sonar data from one location (likelihood)
 - **Updated belief:** Combining prior knowledge with new data (posterior)
-

Bayes' Rule

The Formula

$$P(A \mid B) = [P(B \mid A) \cdot P(A)] / P(B)$$

In words:

$$\text{Posterior} = (\text{Likelihood} \times \text{Prior}) / \text{Evidence}$$

Extended Form

$$P(A \mid B) = [P(B \mid A) \cdot P(A)] / [P(B \mid A) \cdot P(A) + P(B \mid \text{not } A) \cdot P(\text{not } A)]$$

The denominator $P(B)$ can be expanded using the **law of total probability**.

Proportional Form (Most Useful!)

Often, we only care about comparing different hypotheses, so we can ignore the denominator:

$$P(A \mid B) \propto P(B \mid A) \cdot P(A)$$

In words: "Posterior is proportional to likelihood times prior"

Then normalize at the end to ensure probabilities sum to 1.

Terminology: Prior, Likelihood, Posterior

Understanding these terms is crucial for working with Bayes' Rule.

Prior: $P(A)$

What it is: Your belief before seeing any data

Example: "Before searching, here's where I think the submarine might be"

```
prior = submarine_pmf # Initial belief about location
```

Properties:

- Represents background knowledge
- Doesn't depend on observations
- Can be "uninformative" (uniform) or "informative" (based on expert knowledge)

Likelihood: $P(B | A)$

What it is: How probable the data is if the hypothesis were true

Example: "If the submarine were at location $[x, y]$, how likely would we see this sonar reading?"

```
likelihood = search_submarine(10, 7) # Sonar data from searching at [10, 7]
```

Important: This is NOT $P(A | B)$! It's the probability of the *data* given the *hypothesis*.

Properties:

- Depends on your measurement/observation model
- Higher likelihood = data is more consistent with this hypothesis
- Not a probability distribution over hypotheses (doesn't sum to 1)

Posterior: $P(A | B)$

What it is: Updated belief after seeing the data

Example: "After seeing the sonar data, here's my updated belief about the submarine location"

```
posterior = (likelihood * prior) / np.sum(likelihood * prior)
```

Properties:

- Combines prior knowledge with new evidence
- Is a valid probability distribution (sums to 1)
- Can become the prior for the next observation

Evidence: P(B)

What it is: Total probability of observing the data (normalizing constant)

$$P(B) = \sum_a P(B \mid A=a) \cdot P(A=a)$$

Example:

```
evidence = np.sum(likelihood * prior)
```

Properties:

- Ensures posterior is a valid probability distribution
- Same for all hypotheses (so often ignored when comparing hypotheses)
- Can be hard to compute in complex models

Why Bayes' Rule Matters

1. Flipping Conditionals

Often we know $P(B|A)$ but want $P(A|B)$:

Example:

- We know: $P(\text{positive test} \mid \text{disease}) = 0.95$ (sensitivity)
- We want: $P(\text{disease} \mid \text{positive test}) = ?$

Bayes' Rule lets us compute this!

2. Updating Beliefs

Science is about updating beliefs based on evidence:

- Start with hypothesis (prior)
- Collect data (likelihood)
- Update hypothesis (posterior)
- Repeat!

3. Handling Uncertainty

Bayes' Rule gives us a principled way to:

- Incorporate prior knowledge
- Quantify uncertainty
- Make optimal decisions under uncertainty

Working with Bayes' Rule

Step-by-Step Process

Problem: Update submarine location belief after sonar search

Step 1: Define the prior

```
prior = submarine_pmf # Shape: (16, 16)
# This represents P(location) before any search
```

Step 2: Obtain the likelihood

```
likelihood = search_submarine(10, 7) # Shape: (16, 16)
# This represents P(sonar data | location)
# For each location, how likely would we get this sonar reading?
```

Step 3: Compute unnormalized posterior

```
unnormalized_posterior = likelihood * prior # Element-wise multiplication
```

Step 4: Normalize to get posterior

```
evidence = np.sum(unnormalized_posterior)
posterior = unnormalized_posterior / evidence
```

Step 5: Verify

```
assert np.isclose(np.sum(posterior), 1.0) # Should be a valid PMF
```

Complete Example

```
import numpy as np
from submarine import submarine_pmf, search_submarine, show_pmf

# Prior belief
prior = submarine_pmf
print(f"Prior probability submarine at [7, 10]: {prior[7, 10]:.6f}")

# Search at location [10, 7] and get sonar data
likelihood = search_submarine(10, 7)
print(f"Likelihood at [7, 10] given search at [10, 7]: {likelihood[7, 10]:.6f}")

# Apply Bayes' Rule
posterior = (likelihood * prior) / np.sum(likelihood * prior)
print(f"Posterior probability submarine at [7, 10]: {posterior[7, 10]:.6f}")

# Visualize
show_pmf(prior, "Prior")
show_pmf(likelihood, "Likelihood")
show_pmf(posterior, "Posterior")
```

Sequential Bayesian Updates

One of the most powerful aspects of Bayes' Rule: you can apply it repeatedly!

Concept

```
Prior1 → Observe data1 → Posterior1
Posterior1 becomes Prior2 → Observe data2 → Posterior2
Posterior2 becomes Prior3 → Observe data3 → Posterior3
...
```

Each posterior becomes the next prior!

Mathematical Form

After observation 1:

$$P(A \mid B_1) \propto P(B_1 \mid A) \cdot P(A)$$

After observation 2:

$$\begin{aligned} P(A \mid B_1, B_2) &\propto P(B_2 \mid A) \cdot P(A \mid B_1) \\ &\propto P(B_2 \mid A) \cdot P(B_1 \mid A) \cdot P(A) \end{aligned}$$

After observations 1, 2, ..., n (assuming conditional independence):

$$P(A \mid B_1, B_2, \dots, B_n) \propto P(A) \cdot \prod_i P(B_i \mid A)$$

Example: Sequential Search

Search a strip of ocean ($y=5$, $x=0$ to 15) one square at a time:


```
# Start with prior
current_belief = submarine_pmf.copy()

# Store all posteriors
posteriors = [current_belief.copy()] # Include prior as first element

# Search each square in the strip y=5, x=0..15
for x in range(16):
    # Get likelihood from searching at [x, 5]
    likelihood = search_submarine(x, 5)

    # Update belief
    unnormalized = likelihood * current_belief
    current_belief = unnormalized / np.sum(unnormalized)

    # Store this posterior
    posteriors.append(current_belief.copy())

# Now posteriors[0] is the prior
# posteriors[1] is posterior after searching [0, 5]
# posteriors[2] is posterior after searching [0, 5] and [1, 5]
# ...
# posteriors[16] is posterior after searching all 16 squares
```

Visualize the evolution:

```
for i, post in enumerate(posteriors):
    if i == 0:
        show_pmf(post, "Prior")
    else:
        show_pmf(post, f"After searching up to [{i-1}, 5]")
```

Key Insight: Notice how the posterior concentrates more and more around the true location as we gather more evidence!

Practical Examples

Example 1: Single Update

Scenario: We have a prior belief, search at $[10, 7]$, and want the posterior.

```
def bayesian_update(prior, likelihood):  
    """  
    Apply Bayes' Rule to update beliefs  
  
    Parameters:  
    -----  
    prior : array  
        Prior probability distribution  
    likelihood : array  
        Likelihood of observations given each hypothesis  
  
    Returns:  
    -----  
    posterior : array  
        Updated probability distribution  
    """  
    # Element-wise multiplication  
    unnormalized = likelihood * prior  
  
    # Normalize  
    evidence = np.sum(unnormalized)  
  
    if evidence > 0:  
        posterior = unnormalized / evidence  
    else:  
        # No evidence, posterior = prior  
        posterior = prior.copy()  
  
    return posterior  
  
# Use it  
prior = submarine_pmf  
likelihood = search_submarine(10, 7)  
posterior = bayesian_update(prior, likelihood)  
  
show_pmf(posterior, "Posterior after search at [10, 7]")
```

Example 2: Multiple Updates

Scenario: Search multiple locations and combine all evidence.

```
# Start with prior
belief = submarine_pmf.copy()

# Search locations
search_locations = [(10, 7), (8, 9), (7, 10)]

for x, y in search_locations:
    likelihood = search_submarine(x, y)
    belief = bayesian_update(belief, likelihood)
    show_pmf(belief, f"After searching {search_locations[:search_locations.index((x, y))]}")

print(f"Final belief after all searches:")
show_pmf(belief, "Final Posterior")

# Find most probable location
max_prob_idx = np.unravel_index(np.argmax(belief), belief.shape)
print(f"Most probable location: {max_prob_idx} with probability {belief[max_prob_idx]}
```

Example 3: Comparing Hypotheses

Scenario: Two theories about submarine location. Which is more supported by data?

```
# Hypothesis 1: Submarine in top half (y < 8)
h1_prior = np.zeros((16, 16))
h1_prior[:, :8] = submarine_pmf[:, :8] / np.sum(submarine_pmf[:, :8])

# Hypothesis 2: Submarine in bottom half (y >= 8)
h2_prior = np.zeros((16, 16))
h2_prior[:, 8:] = submarine_pmf[:, 8:] / np.sum(submarine_pmf[:, 8:])

# Get data
likelihood = search_submarine(10, 7)

# Update both hypotheses
h1_posterior = bayesian_update(h1_prior, likelihood)
h2_posterior = bayesian_update(h2_prior, likelihood)

# Compare total probability mass
h1_prob = np.sum(h1_posterior)
h2_prob = np.sum(h2_posterior)

print(f"P(top half | data): {h1_prob:.4f}")
print(f"P(bottom half | data): {h2_prob:.4f}")

# Bayes factor: ratio of posteriors
bayes_factor = h1_prob / h2_prob
print(f"Bayes factor (H1 vs H2): {bayes_factor:.2f}")
```

Example 4: Predictive Distribution

Scenario: Where should we search next for maximum information gain?

```
# Current belief (posterior)
current_belief = submarine_pmf # Or result from previous updates

# For each possible search location, compute expected information gain
# (We'll cover this more in the entropy section)

# For now, simple heuristic: search where we're most uncertain
# Uncertainty  $\propto$  entropy  $\propto$   $-p \log(p)$ 

uncertainty = -current_belief * np.log(current_belief + 1e-10)
uncertainty_map = np.sum(uncertainty) # Total uncertainty

# Find location with highest current probability
next_search = np.unravel_index(np.argmax(current_belief), current_belief.shape)
print(f"Search next at: {next_search}")
```

Common Pitfalls

Pitfall 1: Confusing Prior and Likelihood

Wrong:

```
# This is backwards!
posterior = (prior * likelihood) / prior # This just gives likelihood!
```

Right:

```
posterior = (likelihood * prior) / np.sum(likelihood * prior)
```

Remember:

- Prior = $P(\text{hypothesis})$

- Likelihood = $P(\text{data} \mid \text{hypothesis})$
- Posterior = $P(\text{hypothesis} \mid \text{data})$

Pitfall 2: Forgetting to Normalize

Wrong:

```
posterior = likelihood * prior # This doesn't sum to 1!
```

Right:

```
unnormalized = likelihood * prior  
posterior = unnormalized / np.sum(unnormalized) # Now sums to 1
```

Pitfall 3: Treating Likelihood as Probability

The likelihood $P(\text{data} \mid \text{hypothesis})$ is NOT a probability distribution over hypotheses!

```
likelihood = search_submarine(10, 7)  
print(np.sum(likelihood)) # This is NOT necessarily 1.0!  
# Likelihood doesn't have to sum to 1  
  
posterior = bayesian_update(prior, likelihood)  
print(np.sum(posterior)) # This SHOULD be 1.0
```

Pitfall 4: Reusing the Same Prior

When doing sequential updates, remember to use the previous posterior as the new prior!

Wrong:

```
prior = submarine_pmf
for x in range(16):
    likelihood = search_submarine(x, 5)
    posterior = bayesian_update(prior, likelihood) # Always using same prior!
```

Right:

```
belief = submarine_pmf.copy()
for x in range(16):
    likelihood = search_submarine(x, 5)
    belief = bayesian_update(belief, likelihood) # Using updated belief
```

Pitfall 5: Numerical Underflow

When multiplying many small probabilities, you can get numerical underflow.

Solution: Work in log space

```
# Instead of: posterior  $\propto$  likelihood  $\times$  prior
log_posterior = np.log(likelihood + 1e-300) + np.log(prior + 1e-300)

# Normalize in log space (log-sum-exp trick)
log_posterior = log_posterior - np.max(log_posterior) # Prevent overflow
posterior = np.exp(log_posterior)
posterior = posterior / np.sum(posterior)
```

The Base Rate Fallacy

A famous application of Bayes' Rule shows why intuition can mislead us.

Medical Test Example

Scenario:

- Disease prevalence: 1% ($P(\text{Disease}) = 0.01$)
- Test sensitivity: 95% ($P(\text{Positive} | \text{Disease}) = 0.95$)
- Test specificity: 90% ($P(\text{Negative} | \text{No Disease}) = 0.90$)

Question: If you test positive, what's the probability you have the disease?

Intuitive (wrong) answer: "95%! The test is 95% accurate!"

Correct answer using Bayes' Rule:

```
# Prior
p_disease = 0.01
p_no_disease = 0.99

# Likelihood
p_pos_given_disease = 0.95
p_pos_given_no_disease = 0.10 # False positive rate = 1 - specificity

# Evidence (total probability of positive test)
p_positive = (p_pos_given_disease * p_disease +
              p_pos_given_no_disease * p_no_disease)
            = 0.95 * 0.01 + 0.10 * 0.99
            = 0.0095 + 0.099
            = 0.1085

# Posterior
p_disease_given_pos = (p_pos_given_disease * p_disease) / p_positive
                    = (0.95 * 0.01) / 0.1085
                    = 0.0876
                    ≈ 8.8%
```

Correct answer: Only ~8.8% chance of having the disease!

Why?: The disease is rare (low prior), so most positive tests are false positives.

Verification Checklist

When applying Bayes' Rule:

```
# ✓ Prior is a valid PMF
assert np.isclose(np.sum(prior), 1.0)
assert np.all(prior >= 0) and np.all(prior <= 1)

# ✓ Likelihood is non-negative (doesn't have to sum to 1)
assert np.all(likelihood >= 0)

# ✓ Posterior is a valid PMF
assert np.isclose(np.sum(posterior), 1.0)
assert np.all(posterior >= 0) and np.all(posterior <= 1)

# ✓ Posterior is proportional to likelihood × prior
assert np.allclose(posterior, (likelihood * prior) / np.sum(likelihood * prior))

# ✓ If likelihood is uniform, posterior equals prior
uniform_likelihood = np.ones_like(prior)
posterior_uniform = bayesian_update(prior, uniform_likelihood)
assert np.allclose(posterior_uniform, prior)

# ✓ If prior is uniform, posterior is proportional to likelihood
uniform_prior = np.ones_like(prior) / np.sum(np.ones_like(prior))
posterior_uniform_prior = bayesian_update(uniform_prior, likelihood)
normalized_likelihood = likelihood / np.sum(likelihood)
assert np.allclose(posterior_uniform_prior, normalized_likelihood)
```

Summary

Bayes' Rule:

$$P(\text{hypothesis} \mid \text{data}) \propto P(\text{data} \mid \text{hypothesis}) \times P(\text{hypothesis})$$
$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

Key Concepts:

1. **Prior:** Belief before seeing data
2. **Likelihood:** Probability of data given hypothesis
3. **Posterior:** Updated belief after seeing data
4. **Evidence:** Normalizing constant (sum of likelihood $\times$ prior)

Key Steps:

1. Define prior $P(\text{hypothesis})$
2. Compute likelihood $P(\text{data} \mid \text{hypothesis})$
3. Multiply: unnormalized posterior = likelihood $\times$ prior
4. Normalize: posterior = unnormalized / sum(unnormalized)
5. Verify posterior sums to 1

Sequential Updates:

- Posterior from step n becomes prior for step $n+1$
- Equivalent to multiplying all likelihoods together (if conditionally independent)

Common Mistakes:

- Confusing $P(A \mid B)$ with $P(B \mid A)$
- Forgetting to normalize
- Not updating the prior in sequential updates
- Treating likelihood as a probability distribution

Remember: Bayes' Rule is just a formula, but it's the foundation of:

- Scientific inference
- Machine learning
- Decision making under uncertainty
- Rational belief updating